```
$ curl -X 'POST' http://localhost:8080/index.php
HTTP request method: POST
$ curl -X 'PUT' http://localhost:8080/index.php
HTTP request method: PUT
```

Now, if you remember the table from earlier, `POST` and `PUT` are methods that are intended to change the state of the resource on the server. If you check the contents of `index.php`, you will note that they haven't changed. Nor do we have any data source backing the PHP code, which might have changed. The HTTP method is fundamentally just a string with a particular meaning in terms of the protocol specification: the web server isn't obliged to change state just because it received a `POST` request, just as it isn't obliged to prevent state change on a `GET` request. It's up to the web server's implementation to honor the contract made by accepting the request: you, dear programmer, need to make sure each `PUT`, `PATCH`, `POST` et al. behaves correctly. If our simple code inside `index.php` isn't capable of correctly handling anything other than a `GET` request, we should probably update it…

```
$method = $_SERVER['REQUEST_METHOD'];
if ($method !== 'GET') {
    print "HTTP request method not allowed: {$method}" . PHP_EOL;
} else {
    print "HTTP request method: {$method}" . PHP_EOL;
}
```

… and now we should get a different response body depending on the method we use:

```
$ curl -X 'PUT' http://localhost:8080/index.php
HTTP request method not allowed: PUT
```

We could even get a little ahead of ourselves and turn our attention to the response message side of the protocol. The `curl` command above should have generated a line like this in the `cli-server` terminal:

```
[...] 172.17.0.1:54154 [200]: PUT /index.php
[...] 172.17.0.1:54154 Closing
```